

LAB10

Data Structure

Lab10

- The deadline for lab10 submission is 21st May at 11:59 pm.
 - If you have any questions, please contact TA through email.
(justin043435@hanyang.ac.kr)
 - Lab scores will be uploaded to the LMS next week.
 - Questions about Lab 10 scores will only be accepted during next lab session.
-
- Code name: {student_id}_p10.c
 - The code will be tested with 5 different input files.
 - Each input file will be worth 20 points
 - Make sure that the output format matches the provided example exactly.

Evaluation criteria

Category	Evaluation	
p10	100	
Total	100	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

Lab 10. Maze with Disjoint Set

- Design and implement a maze generator using the **Disjoint Set (Union-Find)**.
- The maze is represented as a grid of $n \times n$ cells and 'n' is provided through input file.
- At each step, a wall is selected at random using a **random number generator**. If removing that wall **does not create a cycle**, it is removed
- To enable deterministic behavior during testing, the random seed is provided through the input file. If not provided, the **current time** is used to initialize the seed.
- Ensure that the maze contains **no cycles**, and that there exists unique path from the entrance (top-left) to the exit (bottom-right)

Lab 10. Maze with Disjoint Set

- The input file provides commands such as:
 - **n s** : where n is the size of the maze ($n \times n$), and s is the seed.
 - **n** : if seed is not provided, the program uses `time(NULL)` for random seed generator.

Lab 10. Maze with Disjoint Set

- Structures

```
typedef struct _DisjointSet{
    int size;
    int *ptr_arr; // parent
} DisjointSets;

typedef struct _PrintDisjointSet{
    int size;
    int *ptr_arr;
    // wall, (if wall exists -> 1, otherwise 0)
} PrintDisjointSets;
```

- Functions

```
void init(DisjointSets *sets, PrintDisjointSets *maze, int n, int seed);
int find(DisjointSets *sets, int x);
void union_(DisjointSets *sets, int i, int j);
void createMaze(DisjointSets *sets, PrintDisjointSets *maze, int n);
void printMaze(PrintDisjointSets *maze, int n);
void freeMaze(DisjointSets *sets, PrintDisjointSets *maze);
```

Lab 10. Maze with Disjoint Set

- **void init(DisjointSets *sets, PrintDisjointSets *maze, int n, int seed)**
 - Initialize the Disjoint Set and wall array.
 - Set the seed for random generation.
- **int find(DisjointSets *sets, int x)**
 - Find the set including the number 'x' and return the representative member of the set.
- **void union_(DisjointSets *sets, int i, int j)**
 - Union two sets.
- **void CreateMaze(DisjointSets *sets, PrintDisjointSets *maze, int n)**
 - Randomly remove walls using union-find to generate a maze that includes a path from Start position (top-left) to End position (bottom-right) **WITHOUT any cycles**.

Lab 10. Maze with Disjoint Set

- **void printMaze(PrintDisjointSets *maze, int n)**
 - Print the resulting maze in ASCII format using '*', '|', and '-' characters.
 - Outer walls must exist.
 - Open the entrance (top-left) and exit (bottom-right).
- **void freeMaze(DisjointSets *sets, PrintDisjointSets *maze)**
 - Free all dynamically allocated memory

Lab 10. Maze with Disjoint Set - Initialization

- **Disjoint Set & Wall array initialization**
 - Cells are numbered from top-left(1) to bottom right (36), in row-major order.

start

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

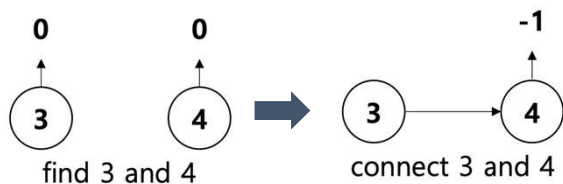
end

- sets->ptr_arr is initialized to 0 for every cell
→ **meaning each is its own root**
- maze->ptr_arr is initialized to 1
→ **meaning that all wall exists**

Lab 10. Maze with Disjoint Set - CreateMaze

- $n = 6$

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36



1. Select a random wall index using random number generator.
2. Calculate the numbers of two cells connected wall
3. Compare the root of cell3 and the root of cell4.
4. If the roots are different, remove the wall and merge two cells.
5. Repeat 1 to 4 until first cell and last cell are in the same set.

How to calculate the cell number from wall number?

For vertical walls (wall 3),

left cell number = 3, right cell number = $4(=3+1)$

For horizontal walls (wall 39),

upper cell number = $3(=39\%(n^2))$, lower cell number = $9(=3+n)$

Lab 10. Maze with Disjoint Set - CreateMaze

- createMaze

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36



1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30
31	32	33	34	35	36

- No cycles!
- First cell and last cell are in the same set!
- Outer walls must exist!

Lab 10. Maze with Disjoint Set - CreateMaze

- **Random number generation**

- Function : `srand()`, `rand()` - `<stdlib.h>` & `time()` - `<time.h>`
- `srand(time(NULL))` // set the random seed using the current time
- `srand(seed)` // set the random seed using given number
- `rand() % N` // generates a random number from 0 to N-1

Lab 10. Maze with Disjoint Set

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct _DisjointSet{
    int size;
    int *ptr_arr; // parent
} DisjointSets;

typedef struct _PrintDisjointSet{
    int size;
    int *ptr_arr;
    // wall, (if wall exists -> 1, otherwise 0)
} PrintDisjointSets;

void init(DisjointSets *sets, PrintDisjointSets *maze, int n, int seed);
int find(DisjointSets *sets, int x);
void union_(DisjointSets *sets, int i, int j);
void createMaze(DisjointSets *sets, PrintDisjointSets *maze, int n);
void printMaze(PrintDisjointSets *maze, int n);
void freeMaze(DisjointSets *sets, PrintDisjointSets *maze);

int main(int argc, char* argv[]){
    int num, seed;
    FILE *fi = fopen(argv[1], "r");

    int inputCount = fscanf(fi, "%d %d", &num, &seed);
    fclose(fi);

    if (inputCount < 2) seed = -1;

    DisjointSets *sets = (DisjointSets*)malloc(sizeof(DisjointSets));
    PrintDisjointSets *maze = (PrintDisjointSets*)malloc(sizeof(PrintDisjointSets));

    init(sets, maze, num, seed);
    createMaze(sets, maze, num);
    printMaze(maze, num);
    freeMaze(sets, maze);

    return 0;
}
```

Lab 10. Maze with Disjoint Set

- Example 1)

```
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > cat input1_seed.txt
3 42
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > ./p10 input1_seed.txt
*-*-*-*
|      |
*  *-*  *
|      |
*-*-*  *
|      |
*-*-*-*
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > ./p10 input1_seed.txt
*-*-*-*
|      |
*  *-*  *
|      |
*-*-*  *
|      |
*-*-*-*
```

Lab 10. Maze with Disjoint Set

- Example 2)

```
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > cat input1_noseed.txt
```

```
3
```

```
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > ./p10 input1_noseed.txt
```

```
*-*-*-*
```

```
  |  |
```

```
* * * *
```

```
| |  |
```

```
*-*-* *
```

```
| | |
```

```
*-*-**
```

```
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > ./p10 input1_noseed.txt
```

```
*-*-*-*
```

```
  |  |
```

```
* * *-*
```

```
|  | |
```

```
* *-*-*
```

```
|
```

```
*-*-**
```

—

Lab 10. Maze with Disjoint Set

- Example 3)

```
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > cat input2_seed.txt  
6 101  
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > ./p10 input2_seed.txt
```

```
*-*-*-*-*  
  |  |  |  
* * * * *  
| |  |  |  
*-*-* * *  
| |  |  |  
* *-* *-*  
| | |  |  
* * * *-*  
|  |  |  |  
*-* *-* *-*  
|  |  
*-*-*-*-*
```

```
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > ./p10 input2_seed.txt
```

```
*-*-*-*-*  
  |  |  |  
* * * * *  
| |  |  |  
*-*-* * *  
| |  |  |  
* *-* *-*  
| | |  |  
* * * *-*  
|  |  |  |  
*-* *-* *-*  
|  |  
*-*-*-*-*
```

Lab 10. Maze with Disjoint Set

- Example 4)

```
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > cat input2_noseed.txt
6
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > ./p10 input2_noseed.txt
*-*-*-*-*
|      |      |
* *-* * * *-*
|      |      |
* *-* *-* * *
|      |      |
* *-*-*-*-*
|      |      |
* *-* *-* *
|      |      |
*-* *-* * *-*
|      |
*-*-*-*-*
[~/Dropbox/자료 구조 /2025자료 구조 /실습 /Lab10/code > ./p10 input2_noseed.txt
*-*-*-*-*
|      |      |
*-* *-* *-* *
|      |      |
* * * * * *-*
|      |      |
* * *-* * * *
|      |      |
*-* *-*-* * *
|      |      |
* * * *-* *-*
|
*-*-*-*-*
```